

A COMPLETE STUDY GUIDE

Testing with Generative AI

Learn it, picture it, explain it. A lesson-style guide for the ISTQB® Certified Tester CT-GenAI exam (Syllabus v1.1).

How this guide teaches

Cornell-style pages. The shaded left column holds cues and keywords; the right holds the explanation. Cover the right side and try to explain each cue from memory.

Analogies and diagrams. Every abstract idea is paired with an everyday comparison or a picture (analogies are my own teaching aids, not syllabus text).

Follow Ana. Ana is a fictional test analyst whose day illustrates each test-task area in Chapter 2.

Check for understanding. Each chapter ends with three short questions and answers.

What the exam expects

Level	You should be able to	Where it shows up
K1 Remember	Recall definitions, lists, names	AI types, defect definitions, mitigation of non-determinism, regulations
K2 Understand	Explain, compare, give examples	Prompt structure, techniques, metrics, risks, architecture, RAG, agents
K3 Apply	Use knowledge on a scenario	Test analysis, design, regression, monitoring; choosing techniques; spotting hallucinations, reasoning errors, bias

Exam prep sources describe about 40 multiple-choice questions in 60 minutes with a pass mark near 65%. Confirm the current rules with your ISTQB® Member Board or exam provider. A Foundation Level certificate is a prerequisite.

Independent study aid written in original wording from the ISTQB® CT-GenAI Syllabus v1.1 (© ISTQB®). The official syllabus remains the authoritative source.

BEFORE YOU START

Your four-week study plan

Roughly 4 to 5 hours per week is enough if you study actively. The syllabus allots 815 minutes of training in total, weighted heavily to prompt engineering.

Week	Read in this guide	Do	Goal
1	Chapter 1 (pp. 3-5) and Ch. 2 up to prompt structure	Explain each cue aloud; try a tokenizer and a multimodal prompt if you have a chatbot	Solid vocabulary
2	Chapter 2 (pp. 6-10)	Write one prompt per test task; classify it with the six components; apply the technique chooser	Confident K3 technique selection
3	Chapter 3 (pp. 11-14) and Chapter 4 (pp. 15-16)	Diagnose invented examples of hallucination, reasoning error and bias; sketch RAG from memory	Risk and architecture fluency
4	Chapter 5 (p. 17), glossary and summary (pp. 18-19)	Take a full practice exam under time; fill the confidence tracker (p. 20)	Close the weak spots

Study techniques that work for this exam

- **Teach it.** Explain hallucination vs reasoning error to a colleague without notes. If you stumble, reread.
- **Make your own examples.** The exam uses scenarios. For every term, invent a testing situation where it appears.
- **Practise with a real tool.** Even 15 minutes with a chatbot per week builds intuition for prompts, temperature effects and non-determinism (the syllabus includes hands-on exercises).
- **Space your reviews.** Revisit each chapter after 1 day, 1 week and 3 weeks.
- **Read question stems for the verb.** Recall, explain, compare, apply and select each ask for different depth.

The big picture in one paragraph

Course in a nutshell

GenAI models (mainly LLMs) predict likely text; that makes them powerful assistants for test tasks and also the source of every risk: they can sound right while being wrong. Testers therefore need to **(1)** understand how LLMs work, **(2)** write good prompts and choose the right technique, **(3)** spot and reduce hallucinations, reasoning errors, bias, privacy and security problems, **(4)** know how LLM-powered test infrastructure fits together (RAG, agents, fine-tuning, LLMops), and **(5)** help their organisation adopt GenAI in a controlled, measured way.

Chapter	Minutes	Theme in five words
1 Introduction	100	How LLMs work, what they offer
2 Prompt engineering	365	Ask well, pick technique, evaluate
3 Managing risks	160	Trust less, verify more, protect data
4 Infrastructure	110	Architecture, RAG, agents, tuning
5 Deployment	80	Adopt safely, measure, change culture

CHAPTER 1 • 100 MIN

Introduction to Generative AI for Software Testing

LOs 1.1.1–1.1.4 and 1.2.1–1.2.2 • mostly K1 and K2

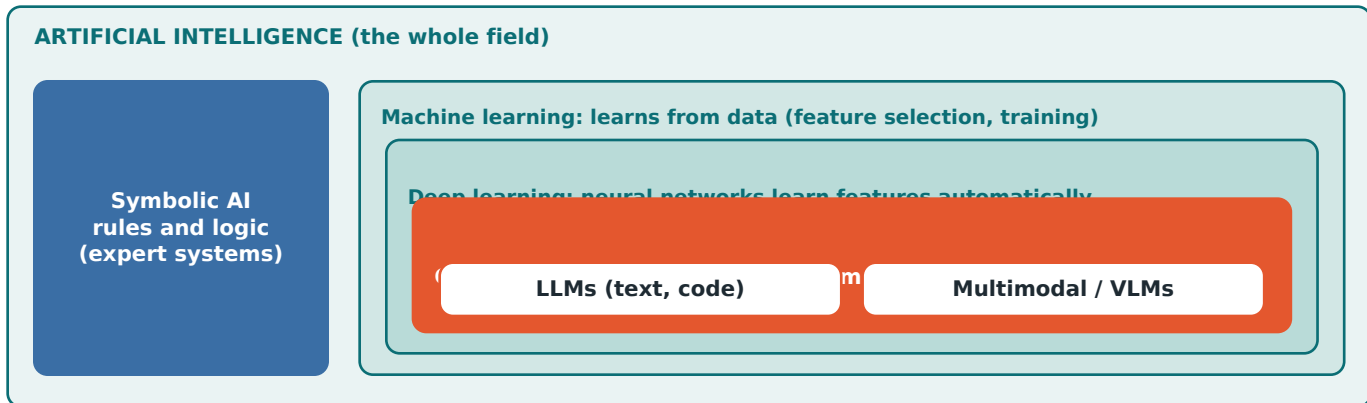


Figure 1. The AI spectrum. Symbolic AI is rule-based; the other layers are data-driven and nested, with generative AI as the creative tip.

Four kinds of AI

symbolic
classical ML
deep learning
generative

Symbolic AI follows rules and logic written by people, like a very detailed decision flowchart. **Classical machine learning** learns patterns from data, but people prepare the data, choose *features* and train the model (e.g. predicting defect-prone modules). **Deep learning** uses many-layered neural networks that discover features on their own from huge, complex data such as images, audio and text. **Generative AI** uses deep learning to *produce new content*, such as text, images or code, by mimicking patterns it saw in training.

Why testers like GenAI: the models come *pre-trained*, so you can apply them to test tasks right away, without a separate training phase. The catch: new risks (Chapter 3).

Inside an LLM

token
embedding
transformer
context window
non-determinism

Tokenization chops text into tokens: characters, word pieces or words. *Analogy: building blocks the model can stack.* **Embeddings** turn tokens into numbers (vectors) that capture meaning and relationships; similar meanings sit close together. *Analogy: a map where "login" and "sign-in" live in the same neighbourhood.*

The **transformer** architecture learns how tokens relate across long sequences. At **inference** the model predicts the *next token*, over and over. This is why output is **plausible but not necessarily correct**.

The **context window** is how much text (in tokens) the model can consider at once. *Analogy: the size of your desk: only what fits is in view.* A bigger window handles long logs and specs but needs more computation and time.

Non-determinism: probabilistic sampling and hyperparameters mean the same prompt can produce different answers.

Model families

foundation
instruction-tuned
reasoning
SLM

Foundation LLM: broad, general, pre-trained on vast data; usually needs adaptation. **Instruction-tuned:** a foundation model further trained on prompt-and-ideal-response pairs so it follows instructions well (often called non-reasoning). **Reasoning LLM:** adds structured multi-step abilities such as chain-of-thought, useful for planning, prioritisation, estimation and other high-load thinking. **SLM:** a small language model, with fewer parameters, light and focused.

Rule of thumb: everyday drafting and formatting → instruction-tuned; complex multi-criteria analysis → reasoning model.

Multimodal text + image VLM wireframes	Multimodal LLMs accept several data types (text, images, sound, video). Each modality is tokenized in its own way; an image, for example, is converted into embeddings by a vision-language model (VLM) before the transformer sees it. In testing you can give the model a user story <i>and</i> a GUI wireframe, ask it to spot mismatches between expected and actual screens, or generate test cases that mix data and visual cues. Hands-on exercises in the syllabus: inspect tokenization, and write a text-plus-image prompt.
What LLMs can do for testers analysis test cases oracles data automation results testware	The syllabus groups typical capabilities into seven areas: • Requirements analysis and improvement: spot ambiguities, inconsistencies and gaps; generate clarifying questions. • Test case creation support: draft cases and objectives from stories. • Test oracle generation: help propose expected results. • Test data generation: datasets, boundaries, combinations. • Test automation support: scripts from descriptions; improvements; technique suggestions. • Test result analysis: summarise results, classify anomalies by severity and priority. • Testware creation and maintenance: test plans, reports, defect reports.
Chatbot vs tool conversation API agents	An AI chatbot is conversational and direct: quick feedback, exploratory work, routine tasks, and usable by non-technical colleagues; you refine results by chaining prompts. An LLM-powered testing application embeds the LLM through APIs into tools and frameworks, so tasks such as test generation, defect analysis and data synthesis become scalable and customisable. The most advanced form uses AI agents (Chapter 4). Either way, output quality depends on prompt engineering.

Common misunderstandings

- ✗ "The LLM looks answers up in a fact database." → ✓ It predicts tokens from learned patterns.
- ✗ "A bigger context window is simply better." → ✓ It also costs computation and time.
- ✗ "GenAI needs a training phase before each test task." → ✓ Pre-trained models can be used directly (though adaptation is possible).

Check for understanding

1. Which AI type needs people to select features and train on prepared data?
2. Why can the same prompt give two different test suites?
3. Which model family suits multi-criteria test prioritisation?

Answers: 1) Classical machine learning. 2) Non-determinism from probabilistic inference and hyperparameters. 3) A reasoning LLM.

CHAPTER 2 • 365 MIN • THE BIG ONE

Prompt Engineering for Effective Software Testing

LOs 2.1.1–2.1.3 (K2), 2.2.1–2.2.5 (K3), 2.3.1–2.3.2 (K2)

ROLE	"Act as a senior test analyst."
CONTEXT	Mobile banking app, password-reset feature
INSTRUCTION	List ambiguities, then write clarification questions
INPUT DATA	User story text + wireframe screenshot
CONSTRAINTS	No invented rules; max 8 questions; mark assumptions
OUTPUT FORMAT	Table: ambiguity impact question

Figure 2. Anatomy of a strong prompt: role, context, instruction, input data, constraints and output format, with a test-analysis example.

Six components R-C-I-I-C-O good input = good output	A prompt is the natural-language input that steers the model. Strong prompts combine six parts: role (persona and tone), context (test object and circumstances), instruction (clear, imperative task), input data (what the model works on: stories, criteria, screenshots, code), constraints (limits and special considerations) and output format (structure of the answer). Missing context is a top cause of weak or invented output.
Three techniques chaining few-shot meta	Prompt chaining: split the work into a series of prompts. After each answer you check and refine before moving on. <i>Analogy: an assembly line with a quality inspector at every station.</i> Zero-, one- and few-shot: no example, one example, or several examples of the desired result. <i>Analogy: showing a new colleague two finished reports before asking for a third.</i> Few-shot gives consistent, well-formatted output. Meta prompting: have the model create or improve prompts, which you then evaluate and refine. <i>Analogy: asking a senior colleague how best to phrase the request.</i> It is a kind of pairing with the AI.
System vs user who sets it? visible?	The system prompt is set up front by the developer or tester. It defines behaviour, tone, domain rules and things to avoid for the whole session, and end users normally cannot see or edit it. The user prompt is what the person types each turn. The model answers using both together. <i>Analogy: the job description versus today's assignment.</i>

Tasks are rarely pure: the three can be combined (meta prompt → add examples → verify in steps).

PROMPT CHAINING	FEW-SHOT	META PROMPTING
Precision matters and a human checks each step	Repetitive task, fixed output format	New or unclear task; help me write the prompt

Figure 3. Choosing a technique (LO 2.2.5).

Meet Ana: GenAI across the test process

Ana is a test analyst on a banking app. Her day maps to the four application areas the syllabus covers (2.2.1 to 2.2.4). Notice that **a human verifies every result**.

2.2.1 Test analysis test basis conditions risk coverage	Morning. Ana gives the model a user story and a wireframe (multimodal input) and asks it to find ambiguities, judge testability and check completeness, one prompt at a time (a chain), correcting the output at each step. Then she asks for test conditions, prioritised by likelihood and impact (payments and login rank high) and checks a requirement-to-condition coverage table for gaps. <i>Typical tasks:</i> identify defects in the test basis; generate test conditions from acceptance criteria; prioritise by risk; support coverage analysis; suggest test techniques (e.g. boundary values, equivalence partitioning).
2.2.2 Design and implementation test cases synthetic data scripts scheduling	Late morning. Ana shows the model two approved Given-When-Then examples (few-shot), then a new story. She asks for synthetic test data that resembles production but contains no real customer details. She turns structured cases into scripts, and asks the model to order the cases by priority and dependencies. Because that is multi-criteria reasoning, she double-checks the order herself. <i>Also useful:</i> a meta prompt to craft an end-to-end procedure prompt; a coverage table produced by the model to verify completeness.
2.2.3 Regression testing keywords impact analysis self-healing reports	Afternoon. Regression suites grow every iteration and run in CI/CD. GenAI helps write keyword-driven scripts, performs impact analysis on code changes to focus on high-risk areas, and enables self-healing scripts that adapt to small UI or API changes (new locators, changed request formats, endpoints or authentication). It also compiles reports with trends and drafts defect reports with logs, screenshots and environment data, and supports root cause analysis. Ana clusters similar failures and keeps a known-anomalies list in one chat.
2.2.4 Monitoring and control metrics dashboards completion	End of day. The model analyses trends, predicts risks, flags deviations from plan and summarises progress for stakeholders in plain language. Its insights help Ana control testing: re-prioritise, adjust schedules, reallocate resources. At the end of the iteration it drafts the completion report and lessons learned.

Exam lens (K3)

Match the story to the technique: **same format over and over** → few-shot; **complex, each step verified** → chaining; **new task, unsure how to prompt** → meta. Real scenarios often mix them: draft a prompt with a meta prompt, add examples (few-shot), then split into checked steps (chaining).

Evaluating results and refining prompts (2.3)

Generated output is only useful if it is measured. The syllabus lists seven metrics for GenAI results on test tasks; a picture in your head helps: imagine ten generated test cases and ask each question below.

Metric	Ask yourself	Testing example
Accuracy	Is it correct overall vs experts, requirements or standards?	Tests cover all specified requirements
Precision	Is it correct for the specific objective?	Generated tests correctly identify anomalies
Recall	Did it find all the relevant things?	Valid and invalid equivalence partitions all covered
Relevance / contextual fit	Does it fit this product and test basis?	Domain rules and test basis respected
Diversity	Are scenarios varied, not repetitive?	Different user behaviours and edge cases
Execution success rate	Do artefacts run as is?	Scripts run with no syntax or format errors
Time efficiency	How much time was saved?	Generation time vs manual effort

Because outputs vary from run to run, judge a prompt on **many runs (statistically relevant data)**, by manual review or automatic comparison against a reference.

Refining prompts

iterate
A/B
analyse
feedback
length

Five ways to improve a prompt: **iterative modification** (adjust wording, add context in steps); **A/B testing** (compare two versions on your metrics); **output analysis** (study errors and inconsistencies and change the prompt to avoid them); **user feedback** (usefulness, clarity, detail); and **adjusting length and specificity** (sometimes more context helps, sometimes shorter is better). Store good prompts in a shared **prompt library** and review them as a team.

Worked example: from weak to strong

Version	Prompt	Problem or improvement
Weak	"Write tests for transfer funds."	No role, context, input data, constraints or format. Likely invented rules, unpredictable layout.
Better	"As a senior test analyst, write functional test cases for the acceptance criteria below (transfer between own accounts in a mobile banking app). Use boundary value analysis, max 10 cases, table format: ID, precondition, steps, expected result." + criteria text	All six components. Still needs verification against the criteria.
Strong (chained)	Prompt 1: find ambiguities. Prompt 2: list testable conditions. Prompt 3: generate cases. Prompt 4: build a coverage table and list gaps.	Checked between steps, reducing hallucinations and reasoning errors.

Reminder: the prioritisation trap

Prioritising with risks and dependencies is a classic reasoning-error area. Verify the order yourself, or use a reasoning model and chain the steps.

Check for understanding

1. Which prompt component would hold "return a table with these columns"?
2. You must turn 200 stories into identically formatted Gherkin. Which technique?
3. Why should one successful run not convince you a prompt is good?

Answers: 1) Output format. 2) Few-shot prompting. 3) Non-determinism: evaluate over many runs on statistically relevant data.

CHAPTER 3 • 160 MIN

Managing Risks of Generative AI in Software Testing

LOs 3.1.1–3.1.4, 3.2.1–3.2.3, 3.3.1, 3.4.1 · K1, K2 and one K3

Three output defects

hallucination
reasoning error
bias

Spot them (K3)

cross-verify
expert review
logic check
run the tests
bias review

Hallucination: output that seems factually wrong or irrelevant to the task, such as invented test cases, non-working scripts or tests for acceptance criteria that do not exist. **Reasoning error**: the model misreads logic (cause and effect, conditions, step-by-step flow) because it matches patterns rather than truly reasoning; watch test planning, effort estimation and prioritisation. **Bias**: output that favours certain information or assumptions, coming from the training data (e.g. mostly English sources), which can skew test data or leave out test types.

All three stem from training data and the transformer's limits. Because of non-determinism a "fixed" defect can **reappear** in another conversation.

Hallucination → cross-verify against documentation and requirements, ask domain experts, check consistency with known behaviour. **Reasoning error** → validate the logic (review cycles, human judgement) and **test the output**: run generated tests or scripts against the test object. **Bias** → check that testware and synthetic data fairly reflect the test strategy and coverage needs, and look for missing test types such as non-functional tests. Spend effort in proportion to the task's risk.

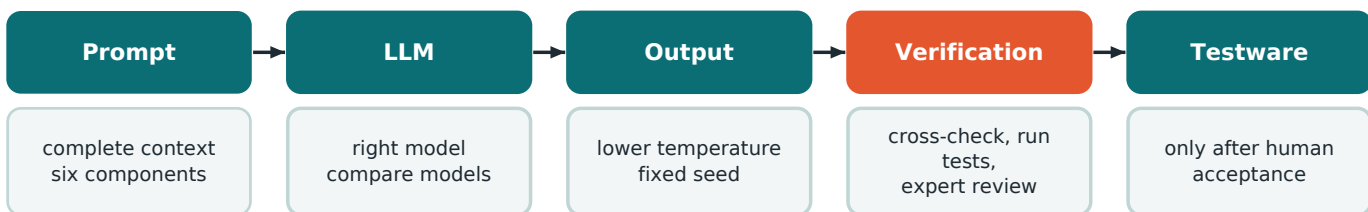


Figure 4. A defence-in-depth pipeline: mitigations applied at each stage before output becomes testware.

Mitigate

context
segments
formats
right model
compare

Give **complete context**; divide work into manageable steps (chaining); use clear, interpretable data formats; choose a fitting model; compare results from several models. Chapter 4 adds RAG and fine-tuning.

Non-determinism (K1): lower the **temperature** to narrow the probability distribution (more consistent, less diverse); set a **random seed** where supported; automate parts of output verification. *Analogy: temperature is a creativity dial; the seed is the same dice roll replayed.* Full reproducibility cannot be guaranteed, especially on long output.

Data privacy and security

Sensitive test data plus a cloud LLM is where most real-world trouble starts. The syllabus separates **privacy** risks (unintended exposure, no control over how a tool stores or uses data, compliance problems such as GDPR) from **security** risks (breaches or unauthorised access to LLM-powered infrastructure, and attackers exploiting the model).

Attack vector	What it is	Example in testing
Context manipulation	Crafted requests to extract confidential training data	Overloading the context window with very long prompts until the model leaks snippets of training data
Request manipulation	Input that disrupts the AI's output	An image that pulls the model into another context and causes hallucinated acceptance criteria
Data poisoning	Manipulating the data models learn from or are evaluated on	Fake ratings when users evaluate an AI-generated test report
Malicious code generation	Tricking an LLM into producing backdoors	A generated script that opens a channel to a malicious IP address

Protect
minimise
anonymise
encrypt
train
audit

Privacy measures: data minimisation (only what you need, and avoid sensitive data unless legally allowed); anonymisation or pseudonymisation; secure storage and transmission with encryption and access control; training and policies for responsible use.

Further strategies: systematic human review of outputs; cross-checking with another LLM; a secure environment matched to confidentiality (commercial secure offering, secure cloud, or a model on your own infrastructure); regular security audits and vulnerability assessments; staying current with best practice. Involve security engineers, legal counsel and the CTO or CISO where they exist.

Story: the shortcut that went wrong

A tester wants "realistic" data and pastes production customer records into a free public chatbot. Three failures at once: **data minimisation** ignored, likely **GDPR** trouble and lack of control over data use, and **shadow AI** (Chapter 5) if the tool is unapproved. The right path: describe the schema and rules, ask for **synthetic** data, or anonymise first, and use an approved environment.

Energy and environment (3.3)

Training and running LLMs uses intensive compute. Energy use grows sharply with usage and depends on task complexity and the computing resources needed: producing an image with a powerful model can use roughly as much energy as fully charging a smartphone, while text uses only a small fraction. Exact figures are hard to pin down, but across millions of users the CO₂ impact is significant. **Good practice:** avoid unnecessary interactions, write concise prompts, use a right-sized model, and do not regenerate for no reason.

Regulations, standards and frameworks (3.4, K1)

Name	Kind	One-line memory
ISO/IEC 42001:2023	Standard	AI <i>management system</i> requirements for organisations
ISO/IEC 23053:2022	Standard	Framework for AI systems using machine learning; lifecycle, data quality, transparency, safety
EU AI Act	Regulation	Classifies AI applications <i>by risk level</i>
NIST AI RMF 1.0	Framework (US)	Guidance on managing AI risks: fairness, transparency, security

Risk response playbook

Situation	First response	Why
Output cites something not in the test basis	Cross-verify with the source; strip or clarify the item	Likely hallucination

Situation	First response	Why
Order, estimate or priority looks illogical	Rebuild as a chain: list criteria, then decide; verify each step	Likely reasoning error
Data or tests all look alike or miss whole test types	Ask explicitly for variety and missing types; review vs strategy	Likely bias
Colleague wants to paste confidential data into a free tool	Stop; minimise or anonymise; use an approved secure environment	Privacy and shadow-AI risk
Results differ on every run	Lower temperature, fix a seed, fix the output format, automate checks	Non-determinism
Generated script does something unexpected	Do not run it in a real environment; review code; report	Possible malicious code generation

Check for understanding

1. Which attack vector uses very long prompts to make the model reveal training data?
2. A generated schedule runs a test before its prerequisite. Which defect?
3. Name two ways to make outputs more repeatable.

Answers: 1) Context manipulation. 2) Reasoning error. 3) Lower the temperature and set a random seed.

CHAPTER 4 • 110 MIN

LLM-Powered Test Infrastructure for Software Testing

LOs 4.1.1–4.1.3 and 4.2.1–4.2.2 • all K2

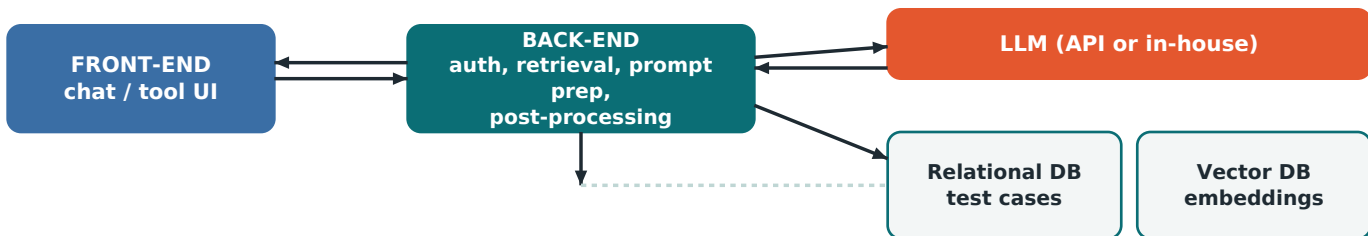


Figure 5. Reference architecture: front-end, back-end, LLM and two kinds of data store.

Architecture

front-end
back-end
LLM
data sources

The **front-end** is where testers type queries or commands. The **back-end** authenticates users, retrieves data, prepares the prompt, talks to the LLM and **post-processes** raw output so it fits the test process. The **LLM** is a third-party service (via API) or an in-house model. **Data sources** include relational databases (structured data such as test cases) and **vector databases** (semantic retrieval using embeddings). Unlike a scripted chatbot, the LLM works as a smart component that reasons over testware and produces insights from context.

A. PREPROCESSING (once, then refreshed)



B. RUNTIME (every question)



Figure 6. Retrieval-Augmented Generation (RAG): index once, retrieve on every question.

RAG

chunk
embed
store
retrieve
generate

RAG boosts relevance and accuracy by adding external data to generation. *Analogy: an open-book exam instead of answering from memory.* Documents are split into chunks (for example 256 to 512 tokens), cleaned, turned into embeddings and stored in a vector database. At runtime the query is embedded, semantically similar chunks are retrieved, and the LLM receives them as context. In testing, RAG lets tools use current specifications, requirements and existing tests, e.g. generate a banking test case using the spec from a vector database and existing tests from a relational one.

Agents

tools
autonomy
multi-agent

An **LLM-powered agent** handles defined tasks semi-autonomously or autonomously, and can *act* by calling predefined functions called **tools**. *Analogy: an intern with a toolbox.* **Autonomous** agents need minimal human intervention; **semi-autonomous** ones get periodic oversight. In **multi-agent** setups, specialised agents coordinate through **orchestration**. Agents can turn stories and requirements into artefacts across analysis, design, implementation, execution and reporting, shifting from script-based to goal-driven automation. They inherit hallucination, reasoning-error and bias risks, so verify results automatically and use semi-autonomous agents for critical work.

Tuning and opsfine-tuning
LLMOps

Fine-tuning further trains a pre-trained model on a targeted dataset so it learns domain knowledge and vocabulary; RAG instead supplies knowledge at query time. **LLMOps** (Large Language Model Operations) covers deploying and managing LLMs. For testing, three routes exist: **AI chatbots**, **GenAI test tools** and **in-house tools**, each with different data-privacy and cost implications (more control usually means more effort and cost).

I summarised 4.2 from official outline material; reread section 4.2 of the syllabus for its exact wording.

Three ways to get better answers

Approach	What changes	Good when	Watch out
Better prompts	The input	Always the first step	Still needs verification
RAG	Context supplied at query time from your data	Answers must follow current specs and existing tests	Depends on quality of indexed documents and retrieval
Fine-tuning	The model itself (extra training on targeted data)	Domain vocabulary or specialised reasoning is needed	Needs suitable data, effort and cost; must be repeated as needs change

Chatbot or agent?

	Chatbot	Agent
Acts on other systems	Mainly answers	Can call tools to act
Human involvement	Every turn	Autonomous (minimal) or semi-autonomous (periodic oversight)
Best practice	Verify answers	Automated verification; semi-autonomous for critical tasks

Check for understanding

1. What does the back-end do besides calling the LLM?
2. List the two RAG runtime steps.
3. Which changes the model itself: RAG or fine-tuning?

Answers: 1) Authentication, data retrieval, prompt preparation and post-processing. 2) Retrieval, then generation. 3) Fine-tuning.

CHAPTER 5 • 80 MIN

Deploying and Integrating GenAI in Test Organizations

Roadmap for adoption and change management



Figure 7. Phases of adopting GenAI in testing.

Shadow AI unapproved risks	Shadow AI means staff use unapproved AI tools without clear guidelines. Risks include data privacy and information security incidents, regulatory and intellectual-property trouble (for example GDPR) and legal consequences. <i>Analogy: company files stored on a personal cloud drive.</i> Provide approved tools, clear policies and training.
Strategy objectives models data measure comply	A GenAI strategy for testing starts with measurable objectives (for example, reduce the time spent writing test cases), chooses compatible LLMs or SLMs, and covers data quality, training, measurement and compliance.
Choosing models capabilities cost hosting licence	Match a model's capabilities, supported inputs, reasoning ability and outputs to the task. Weigh benchmarks and evaluation results, licensing, cost, hosting options, performance and fine-tuning potential, and run pilots to compare.
Change management skills capability process	People need prompt-engineering skills, critical evaluation of AI output, awareness of privacy and security, and solid testing fundamentals. Build capability incrementally with hands-on practice and shared prompt libraries, and update processes with review and verification steps and clear roles. <i>Section 5.2 is summarised at a high level; consult the syllabus for its exact lists.</i>

Check for understanding

1. Name the three adoption phases in order.
2. What is shadow AI and one risk?
3. Why run a pilot before rollout?

Answers: 1) Discovery, initiation, usage definition. 2) Unapproved AI tool use; e.g. data privacy breach or IP/regulatory issues. 3) Compare models, check data quality and compliance, measure benefits.

TOOLKIT

Prompt library: annotated templates

Replace the [brackets] with your own details. Each template shows which components and technique it uses; my own examples for study, not syllabus text.

1. Ambiguity finder Chaining, step 1 of a refinement chain	Act as a senior test analyst. Context: we test [feature] of [product]. Review the user story and acceptance criteria below and list ambiguities, contradictions and missing information. Do not invent rules; mark assumptions. Return a table: item why it matters question for the product owner. [paste story and criteria] Why it works: Role, context, instruction, input data, constraints, output format. Next steps: evaluate testability, then completeness, verifying each result.
2. Risk-prioritised test conditions Chaining, after the criteria are verified	Using the verified acceptance criteria below, derive test conditions. Rank each High/Medium/Low by likelihood and impact, mentioning regulatory or user-facing aspects. Return: ID condition linked criterion risk rationale. [paste criteria] Why it works: Supports test analysis and coverage. Verify the ranking; prioritisation is reasoning-heavy.
3. Gherkin few-shot Few-shot prompting	You write Given-When-Then scenarios. Follow exactly the style of the two examples. Example 1: [story → conditions → scenarios]. Example 2: [story → conditions → scenarios]. Now do the same for: [new story]. Why it works: Best for repetitive tasks with a constrained format. If output drifts, refine the examples, not just the wording.
4. Synthetic test data Structured single prompt	Generate [n] synthetic records for [entity] matching this schema: [fields and rules]. Include boundary values and invalid cases marked as such. Do not reproduce real personal data. Output as CSV with a header row. Why it works: Privacy-preserving data. Check rule compliance and diversity; never paste production records.
5. Coverage check Chaining verification step	Below are acceptance criteria and generated test cases. Build a coverage table: criterion covering test IDs gap? List criteria with no tests and propose the missing cases. [paste both] Why it works: Detects gaps and hallucinated links; spot-check the mapping yourself.
6. Regression report analysis Chaining inside one conversation	Step 1: compare the test results with the specification and list failures. Step 2: cluster similar failures. Step 3: check each cluster against the known-anomalies list [paste list] and mark new ones. Ask me before assuming anything missing. Why it works: Multi-step with an evolving list; keep everything in one chat so context builds.
7. Meta prompt Meta prompting	I need a reusable prompt that makes an LLM [goal]. Ask me any clarifying questions first. Then write the prompt with role, context, instruction, input data, constraints and output format, and explain each part briefly. Why it works: Evaluate the produced prompt on sample data, refine, then store it in the team prompt library.
8. Assistant system prompt System prompt	You are a testing assistant aligned with ISTQB terminology. Use formal, concise language. Never invent requirements; if information is missing, say so and ask. Cite the testing principle or technique you apply. Refuse to process personal or confidential data. Why it works: Set once by the developer or tester; defines behaviour for the whole session.

EXAM SKILLS

How to read exam questions

Follow the four-step method: (1) find the verb (recall, explain, apply, select), (2) find the key noun (technique, risk, metric, component), (3) eliminate absolutes, (4) choose the option that best fits the *scenario*, not merely a true statement.

Walkthrough 1 (K3, technique)	"A team must produce 60 API test scripts with the same keyword library and layout. Which technique is BEST?" Options: (a) zero-shot with no format, (b) few-shot with sample scripts, (c) increasing the temperature, (d) meta prompting only. Reasoning Verb: select. Key idea: repetitive work with a fixed format → few-shot. (a) has no format guidance; (c) randomness; (d) helps write prompts but does not pin the layout. Answer: (b).
Walkthrough 2 (K3, defect)	"The LLM lists a test for a two-factor rule that appears nowhere in the specification." Options: hallucination, reasoning error, bias, data poisoning. Reasoning Verb: identify. Invented content unrelated to the test basis → hallucination. Reasoning error needs fault, bias needs skew; poisoning is an attack. Answer: hallucination.
Walkthrough 3 (K2, explain)	"Why does a larger context window not always help?" Options: it removes tokens; it increases computation and processing time; it lowers accuracy of embeddings; it disables system prompts. Reasoning Verb: explain. The syllabus trade-off is more computational complexity and time. The other options involve accuracy. Answer: more computation and processing time.
Walkthrough 4 (K2, absolute wording)	"Which statement is correct? (a) RAG guarantees correct answers, (b) RAG grounds answers by retrieving relevant chunks, (c) RAG retrains the model, (d) RAG removes the need for prompts." Reasoning Absolutes ("guarantees", "removes the need") and "retrains" are red flags. Answer: (b).

Time and nerves

About 90 seconds per question. Answer certain items first, flag long scenarios and return. Never leave an item blank. Reread the last sentence of every scenario before choosing.

PRACTICE

Scenario clinic

Read each situation, decide what is happening, then read the coaching note.

1. The phantom requirement	An LLM generates ten test cases for "Reset password". Test 7 checks "lock account after 3 wrong answers to the security question". No such rule exists in the spec supplied. Coach Hallucination. Cross-verify against the spec; provide complete context; consider RAG; keep human review. Remove test 7 or clarify the rule with the product owner.
2. The impossible schedule	Given 12 cases with dependencies, the model puts the "refund" test before the "purchase" test it depends on. Coach Reasoning error. Validate the logic; chain the steps (list dependencies, then order); use a reasoning model or compare models.
3. The same-looking users	All generated customer data uses names from one region, and no accessibility or performance tests are proposed. Coach Bias. Ask explicitly for diverse data and non-functional tests; review testware against the test strategy.
4. Two hundred stories	A team must convert 200 user stories into the same Given-When-Then layout. Coach Few-shot prompting with a few approved examples. Add chaining to verify batches if quality matters.
5. Daily reports keep changing shape	A nightly prompt gives a differently structured report every day. Coach Non-determinism: lower temperature, set a seed if supported, use a fixed output format with an example and automate verification.
6. The overloaded prompt	A tester sends extremely long inputs and the model starts printing unrelated fragments of what look like other people's documents. Coach Context manipulation (attempting to extract training data). Apply security controls, audits, and review. Do not send sensitive data.
7. A scripted regression suite keeps breaking	Locators change every sprint, and 30% of failures are false alarms. Coach Self-healing tests and impact analysis from GenAI can adapt scripts and target high-risk areas; still need human review of results.

REFERENCE

Plain-language glossary

Term	Meaning	Term	Meaning
A/B testing of prompts	Comparing two prompt versions on the same metrics.	Meta prompting	Using the LLM to craft or improve prompts.
AI chatbot	A conversational interface to an LLM.	Multimodal model	Handles text, images, sound or video.
Bias	Skewed output caused by training data.	Non-determinism	Same input, possibly different output.
Chain-of-thought	Step-by-step intermediate reasoning used by reasoning LLMs.	Prompt chaining	A series of verified prompts that break a task down.
Context window	How many tokens the model can consider at once.	RAG	Retrieval-augmented generation: ground answers in retrieved data.
Data poisoning	Corrupting the data a model learns from or is judged on.	Reasoning error	Faulty logical conclusion.
Deep learning	Neural networks that learn features automatically.	Seed	Value that makes pseudo-random behaviour repeatable.
Embedding	Numeric vector capturing a token's or chunk's meaning.	Shadow AI	Unapproved AI tool use.
Few-shot prompting	Giving several examples in the prompt.	SLM	Small language model.
Fine-tuning	Extra training on targeted data to specialise a model.	System prompt	Behaviour rules for a session, set by developer/tester.
Foundation LLM	Broad pre-trained base model.	Temperature	Randomness setting; lower means steadier output.
Hallucination	Wrong or irrelevant output that sounds convincing.	Token	Unit of text (character, sub-word or word).
Instruction-tuned LLM	Model trained on prompt-response pairs to follow instructions.	Vector database	Store for embeddings, searched by similarity.
LLM-powered agent	LLM that can call tools and act semi- or fully autonomously.	VLM	Vision-language model combining images and text.
LLMOps	Operations for deploying and managing LLMs.		

Ten things candidates commonly miss

- Lowering temperature helps **consistency**; raising it helps **variety**.
- Plausible ≠ correct: LLMs predict tokens, they do not verify facts.
- Few-shot needs **more than one** example; one is one-shot.
- System prompts are usually **invisible** to end users.
- Precision and recall are different; recall means finding **all** relevant items.
- Metrics need **many runs** because of non-determinism.
- Output testing (running the generated scripts) is a way to catch reasoning errors.
- RAG retrieves at query time; fine-tuning changes the model.
- Semi-autonomous agents are preferred for **critical** tasks.
- Image generation can consume far more energy than text.

REVIEW

One-page summary map

Chapter	Must-know items	Common exam framing
1 Foundations	Four AI types; token, embedding, transformer, context window; foundation / instruction-tuned / reasoning; multimodal and VLM; seven capabilities; chatbot vs application	K1 list; K2 "explain why" (non-determinism, plausible not correct); K2 choose a model family
2 Prompting	Six components; chaining, few-shot, meta; system vs user prompt; test analysis, design, regression, monitoring tasks; seven metrics; five refinement techniques	K3 select a technique; K2 identify a component; K2 metric definitions
3 Risks	Hallucination, reasoning error, bias; detection and mitigation; temperature and seed; privacy and security; four attack vectors; energy; four standards	K3 diagnose a defect; K2 attack vector match; K1 regulation match
4 Infrastructure	Front-end, back-end, LLM, relational and vector DB; RAG (chunk, embed, store, retrieve, generate); agents and tools; fine-tuning; LLMOps approaches	K2 explain a flow; K2 RAG vs fine-tuning
5 Deployment	Shadow AI; strategy with measurable objectives; model selection factors; discovery, initiation, usage definition; skills and change	K2 recognise risk; K1 order of phases

Memory story

Imagine **Ana** opening a chatbot (Ch.1). She writes a **six-part prompt** (Ch.2), uses **chaining** to check each step, and **few-shot** examples to keep the format. She distrusts anything plausible-but-unchecked (**hallucination, reasoning error, bias**), keeps **secrets out** of public tools (Ch.3), and her company builds a **RAG** system with a **vector database** so answers come from real specs (Ch.4). Finally leadership runs **discovery, initiation, usage definition** and bans **shadow AI** (Ch.5).

Self-explain prompts (answer aloud)

- Explain to a non-technical manager why an AI-written test suite still needs human review.
- Describe, in four sentences, how a RAG-based test assistant answers a question.
- Give one testing example each for hallucination, reasoning error and bias.
- Choose and justify a prompting technique for: (a) regression report analysis; (b) 200 Gherkin scenarios; (c) a complex requirement with sign-off after each step.
- List four mitigations you would insist on before letting testers use a public chatbot.

Last-week priorities

- Days 1-2: chapters 2 and 3 tables and mnemonics.
- Days 3-4: timed practice questions; log misses by chapter.
- Day 5: redo misses; explain each aloud.
- Day 6: skim the glossary and summary map.
- Day 7: rest, sleep, arrive early.

Confidence tracker and notes

Topic	Week 1	Week 2	Week 3	Week 4
AI types and LLM basics	☐ __	☐ __	☐ __	☐ __
Model families and multimodal	☐ __	☐ __	☐ __	☐ __
Test-task capabilities; chatbot vs tool	☐ __	☐ __	☐ __	☐ __
Prompt components	☐ __	☐ __	☐ __	☐ __
Chaining, few-shot, meta, system vs user	☐ __	☐ __	☐ __	☐ __
Test analysis and design with GenAI	☐ __	☐ __	☐ __	☐ __
Regression, monitoring and control	☐ __	☐ __	☐ __	☐ __
Metrics and prompt refinement	☐ __	☐ __	☐ __	☐ __
Hallucination, reasoning error, bias	☐ __	☐ __	☐ __	☐ __
Non-determinism and temperature	☐ __	☐ __	☐ __	☐ __
Privacy and security attack vectors	☐ __	☐ __	☐ __	☐ __
Energy and regulations	☐ __	☐ __	☐ __	☐ __
Architecture and RAG	☐ __	☐ __	☐ __	☐ __
Agents, fine-tuning, LLMOps	☐ __	☐ __	☐ __	☐ __
Shadow AI, strategy, model selection, adoption phases	☐ __	☐ __	☐ __	☐ __

[illegible]

Aim for understanding, not memorisation: GenAI is plausible, not guaranteed. If you can explain why, and what a tester should do about it, you are ready.